❖ Python DB and Framework

(1) HTML in Python

Q-1) Introduction to embedding HTML within Python using web frameworks like Django or Flask.

- In Python web frameworks like Django and Flask, you embed and render HTML primarily by using a template engine (Jinja2 for Flask, the Django Template Language for Django) and the framework's render_template or render function, respectively. This approach allows for dynamic content generation and separation of concerns.

- **Implementation in Flask**

- Flask uses the Jinja2 template engine.

(1) Project Structure

- Create a folder named templates in the same directory as your Python application file (app.py).

```
/your_project

|-- app.py
|-- /templates
|   |-- index.html
```

(2) Python Code (app.py)

- Import and use the render_template function in your view function to specify the HTML file to load.

```
from flask import Flask, render_template

app = Flask(__name__)

@app.route('/')
def home():
    # You can pass variables to the template here
    user_name = "Alice"
    return render_template('index.html', name=user_name)

if __name__ == '__main__':
    app.run(debug=True)
```

(3) HTML Code (templates/index.html)

- Use double curly braces {{ variable_name }} to embed Python variables within the HTML.

```
<!DOCTYPE html>
<html>
<head>
    <title>Flask Intro</title>
```

```
</head>
<body>
    <h1>Hello, {{ name }}!</h1>
    <p>This content is from a rendered HTML template.</p>
</body>
</html>
```

- **Implementation in Django**

➢ Django uses its own Django template language.

(1) Project Structure

➢ By default, Django looks for templates in a templates subdirectory within your app's directory.

```
/your_project

|-- manage.py
|-- /your_app
|   |-- views.py
|   |-- /templates
|   |   |-- index.html
```

(2) Python Code (views.py)

- Import render from django.shortcuts and use it in your view function.

```
from django.shortcuts import render


def index_view(request):
    # Data to pass to the template
    context = {'greeting': 'Welcome to my Django app!'}
    return render(request, 'index.html', context)
```

(3) HTML Code (templates/index.html)

- Similar to Flask, use {{ variable_name }} for variables and {% tag_name %} for logic/tags.

```
<!DOCTYPE html>
<html>
<head>
    <title>Django Intro</title>
</head>
<body>
    <h1>{{ greeting }}</h1>
    {% if greeting %}
        <p>The greeting is available.</p>
    {% endif %}
</body>
</html>
```

Q-2) Generating dynamic HTML content using Django templates.

- Generating dynamic HTML content in Django involves using the Django Template Language (DTL) to mix static HTML with dynamic data passed from Django views.

- **Core Concepts**

- **Views:** Python functions or classes that retrieve data (e.g., from a database) and pass it to the template.
- **Templates:** Text files (usually .html) containing static parts of the output and special DTL syntax.
- **Context:** A dictionary that holds the dynamic data passed from the view to the template.

- **DTL Syntax:**

- **Variables:** {{ variable_name }} are placeholders for dynamic content from the context.

- **Tags:** {% tag_name %} control the logic and flow, such as loops, conditionals, and inheritance.

- **Filters:** {{ variable_name|filter_name }} modify the display of variables.

- **Step-by-Step Guide**

(1) Configure Template Settings

- In your settings.py file, ensure the TEMPLATES setting is correctly configured to find your templates. APP_DIRS: True is usually set by default, allowing Django to look for a templates directory within each app.

```python
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [], # Can add project-level template directories here
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```

(2) Create a Template File

- Inside your app directory, create a templates folder, and within that, an HTML file (e.g., myapp/templates/home.html).

```
html
<!-- myapp/templates/home.html -->
<!DOCTYPE html>
<html>
<head>
    <title>Welcome Page</title>
</head>
<body>
    <h1>Hello, {{ user_name }}!</h1>
    <p>Today's date is: {{ current_date|date:"F d, Y" }}</p>

    <h2>My Favorite Things:</h2>
    <ul>
    {% for item in items %}
        <li>{{ item }}</li>
    {% empty %}
        <li>No items found.</li>
    {% endfor %}
    </ul>
</body>
</html>
```

(3) Define a View Function

- In your app's views.py, fetch the dynamic data and use the render() shortcut to pass it to the template via a context dictionary.

```python
# myapp/views.py
from django.shortcuts import render
from datetime import datetime

def home_view(request):
    context = {
        'user_name': 'Alice',
        'current_date': datetime.now(),
        'items': ['Coding', 'Reading', 'Hiking']
    }
    return render(request, 'home.html', context)
```

(4) Map a URL to the View

- In your app's urls.py, link the view to a specific URL pattern.

```python
# myapp/urls.py
from django.urls import path
from . import views
```

```
urlpatterns = [
    path('', views.home_view, name='home'),
]
```

(2) CSS in Python

Q-1) Integrating CSS with Django templates.

- Integrating CSS with Django templates is primarily handled through Django's **static files mechanism**. This involves configuring settings, organizing your CSS files in a specific directory structure, and using the {% load static %} template tag in your HTML.

(1) Configure settings.py

- In your project's settings.py file, ensure that the django.contrib.staticfiles app is included in INSTALLED_APPS (it is by default in new projects) and that STATIC_URL is defined.

python
```
# my_project/settings.py

INSTALLED_APPS = [
    # ... other apps
    'django.contrib.staticfiles',
]

STATIC_URL = '/static/'

# Optional: Define additional directories for static files outside of app folders
STATICFILES_DIRS = [
    BASE_DIR / 'mystaticfiles', # Example: a 'mystaticfiles' folder in the project root
]
```

(2) Organize your CSS files

- Django can find static files within a static folder inside any of your apps, or in the directories specified in STATICFILES_DIRS.
  A recommended structure for an app named my_app is to namespace your static files to prevent naming conflicts with other apps:

```
my_project/
├── manage.py
├── my_project/
│   ├── settings.py
│   └── ...
├── my_app/
│   ├── static/
│   │   ├── my_app/
│   │   │   ├── css/
│   │   │   │   └── style.css
│   │   │   └── img/
│   │   └── ...
│   └── templates/
│   └── ...
└── mystaticfiles/
    └── ...
```

(3) Link the CSS in your Django template

- In your HTML template file
  (e.g., my_app/templates/my_app/index.html), use the {% load static %} template tag at the top of the file and the {% static %} tag to build the URL for your CSS file:

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>My Page</title>
    <!-- Link the CSS file using the static tag -->
    <link rel="stylesheet" href="{% static 'my_app/css/style.css'
%}">
</head>
<body>
    <h1>Welcome!</h1>
    <!-- ... rest of your HTML ... -->
</body>
</html>
```

(4) Development and Production Differences

- **Development (DEBUG=True)**: The Django development server (python manage.py runserver) automatically serves static files with minimal configuration.

- **Production (DEBUG=False)**: You need to run python manage.py collectstatic to gather all static files into the directory specified by the STATIC_ROOT setting. A production web server (like Nginx or Apache) is then responsible for serving these files efficiently.

Q-2) How to serve static files (like CSS, JavaScript) in Django.

- Serving static files in Django (CSS, JavaScript, images) requires different configurations for development and production environments. In development, Django handles files automatically, while in production, a dedicated web server is used.

(1) <u>Configuration in settings.py</u>

- Ensure django.contrib.staticfiles is in your INSTALLED_APPS (it is by default in new projects).

  - **STATIC_URL**: Defines the base URL from which static files will be served.

    ```
    STATIC_URL = '/static/'
    ```

  - **STATICFILES_DIRS**: A list of directories where Django's collectstatic command will look for extra static files, in addition to app-specific static/ folders.

    ```
    import os
    from pathlib import Path

    BASE_DIR = Path(__file__).resolve().parent.parent # Example of obtaining BASE_DIR

    STATICFILES_DIRS = [
        os.path.join(BASE_DIR, 'static'),
    ]
    ```

  - **STATIC_ROOT**: (For production only) The absolute path to the directory where all static files will be collected by the collectstatic command. **Do not** put static files in this directory during development.

```
STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles')
```

(2) File Organization

- **App-specific files**: Place files in a static/ folder within your app (e.g., my_app/static/my_app/style.css). Namespacing them with an additional app-specific directory is a best practice to prevent naming conflicts.
- **Project-wide files**: Place general files in the directory specified by STATICFILES_DIRS (e.g., project_root/static/css/main.css).

(3) Usage in Templates

- Use the {% static %} template tag to reference static files.

- Load the static files library at the top of your template:

```
{% load static %}
```

- Link to your files:

```
<link rel="stylesheet" type="text/css" href="{% static
'my_app/style.css' %}">
<script src="{% static 'js/main.js' %}"></script>
<img src="{% static 'images/logo.png' %}" alt="Logo">
```

(4) Serving in Development

- When DEBUG = True in settings.py, the Django development server (run with python manage.py runserver) automatically serves static files from all necessary locations.

(5) Serving in Production

- In production, DEBUG should be False, and Django will *not* serve static files. A dedicated web server (like Nginx or Apache) or a third-party service (like a CDN or Amazon S3) is required for performance and security.

(1) Collect files: Run the collectstatic command to gather all static files into the STATIC_ROOT directory:

```
python manage.py collectstatic
```

(2) Configure a web server: Configure your web server (e.g., Nginx) to serve files from the STATIC_ROOT directory at the STATIC_URL.

(3) Alternative (WhiteNoise): For simpler deployment (e.g., on platforms like Heroku that don't easily allow dedicated web server config), consider using the WhiteNoise package to serve static files directly from the Django application, with support for caching and compression.

(3) JavaScript with Python

Q-1) Using JavaScript for client-side interactivity in Django templates.

- To implement client-side interactivity using JavaScript in Django templates, you primarily link an external JavaScript file to your template and use mechanisms to pass data securely between Django's server-side context and the client-side JavaScript.

(1) Linking a Static JavaScript File

- The standard method is to manage your JavaScript files as static files within your Django project.

- **Create a static folder** within your app directory (e.g., members/static/members/my_script.js).

- **Load static files** in your template using the {% load static %} template tag.

- **Include the JavaScript file** using a <script> tag with the {% static %} tag for the source URL.

- **Example Template Code:**

```
{% load static %}

<!DOCTYPE html>
<html>
<head>
    <title>Django JS Integration</title>
</head>
<body>
```

```
    <button onclick="myFunction()">Click me for client-side
interactivity!</button>

    <!-- Link the external JavaScript file -->
    <script src="{% static 'members/my_script.js'
%}"></script>
</body>
</html>
```

- **Example JavaScript File (my_script.js):**

```
function myFunction() {
    alert("Hello from a static JavaScript file!");
}
```

(2) Passing Data from Django to JavaScript

- To make server-side data available to client-side JavaScript, the most secure and recommended way is to use the built-in json_script template tag. This prevents XSS vulnerabilities by safely outputting data into a non-executable script tag.

- In your Django **view**, pass a queryset or dictionary to the template context.

- In your **template**, use |json_script:"element-id" to render the data safely into a <script type="application/json"> element.

- In your **JavaScript**, retrieve the data using JSON.parse(document.getElementById('element-id').textContent).

- **Example Django View:**

```
from django.shortcuts import render
import json
from django.http import JsonResponse

def my_view(request):
    data = {'username': request.user.username,
'isAuthenticated': request.user.is_authenticated}
    return render(request, 'my_template.html', {'user_data':
data}
```

- **Example Template Snippet:**

```
...
<script id="user-data" type="application/json">
    {{ user_data|json_script:"user-data" }}
</script>

<script src="{% static 'members/my_script.js' %}"></script>
...
```

- **Example JavaScript Snippet (my_script.js):**

```
// Retrieve the data from the script tag

const userData =
JSON.parse(document.getElementById('user-
data').textContent);

if (userData.isAuthenticated) {
    console.log(`Welcome back, ${userData.username}!`);
    // Add client-side logic based on the data
}
```

(3) Communicating from JavaScript back to Django

- For dynamic updates without full page reloads (e.g., submitting a form, fetching new content), use client-side **AJAX** (Asynchronous JavaScript and XML) requests, typically using the Fetch API or libraries like jQuery.

- JavaScript makes an HTTP request (e.g., POST, GET) to a specific Django URL endpoint (an API view).

- The Django view processes the request, interacts with the database, and returns data, often in JSON format, using a JsonResponse.

- JavaScript receives the response and updates the page's DOM accordingly.

Q-2) Linking external or internal JavaScript files in Django.

- Linking JavaScript files in Django involves configuring Django's **static files** handling and using specific template tags in your HTML files. The process works for both external library files and your own internal project-specific files.

  - **<u>Setup Static Files in Django</u>**

- Before linking, ensure your Django project is configured to manage static files:

  (1) Create a static folder: In your app directory, create a folder named static. Inside that, it is a good practice to create another folder with your app's name to avoid naming conflicts (e.g., app_name/static/app_name/js/). Alternatively, you can have a project-level static folder.

  (2) Configure settings.py: Ensure your settings.py file has django.contrib.staticfiles in INSTALLED_APPS (it is usually there by default). Define STATIC_URL and STATICFILES_DIRS if you are using a project-level static folder:

```
python
STATIC_URL = '/static/'
STATICFILES_DIRS = [
    BASE_DIR / "static",
    # other static dirs
]
```

  (3) Place your JavaScript files: Put your .js files in the appropriate static subfolder (e.g., static/js/my_script.js).

- **<u>Linking in HTML Templates</u>**

➢ In your Django HTML template, you link the JavaScript file using the {% load static %} template tag and the {% static %} tag within a <script> tag:

(1) Load the static files library: Add {% load static %} at the top of your HTML template file.

(2) Link the file: Use the {% static %} tag in the src attribute of the <script> tag:

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>My Django App</title>
</head>
<body>
    <!-- Your content -->

    <!-- Link to your external JavaScript file -->
    <script src="{% static 'js/my_script.js' %}"></script>
</body>
</html>
```

➢ Placing scripts at the end of the <body> section, just before the closing </body> tag, is a recommended practice for better page load performance.

- **Passing Django Data to JavaScript**

➢ You cannot use Django template tags directly inside an external .js file because the browser caches the static files and the Django template engine does not process them. To pass dynamic data from a Django view to an external JavaScript file, use one of these methods:

- **HTML data attributes:** Store the data in a data attribute of an HTML element, then access that attribute from your external JS file:

```
<!-- In your Django template -->
<div id="my-element" data-my-var="{{ django_variable
}}"></div>
<script src="{% static 'js/my_script.js' %}"></script>
```

- **Global JavaScript variable:** Define a global JavaScript variable in a <script> tag in your template *before* loading the external script that uses it:

```
<!-- In your Django template -->
<script>
   var myVar = "{{ django_variable }}";
</script>
<script src="{% static 'js/my_script.js' %}"></script>
```

- **json_script filter:** Use the | json_script:"element-id" filter in your template to safely output a JSON representation of your data into a script tag. Your external JavaScript can then parse this data by accessing the element's text content:

```
<!-- In your Django template -->
{{ django_data|json_script:"county-data" }}
<script src="{% static 'js/my_script.js' %}"></script>
```

- In my_script.js:

```
var countyData =
JSON.parse(document.getElementById('county-
data').textContent);
// Use countyData
```

(4) Django Introduction

Q-1) Overview of Django: Web development framework.

- Django is a high-level, open-source web framework written in Python that encourages **rapid development** and **clean, pragmatic design**. It follows a "batteries-included" philosophy, providing ready-to-use components for common web development tasks, allowing developers to focus on unique application features.

- **Components of Django**

Component	Description
Models	Define database structure
Views	Handle request and business logic
Templates	Generate HTML pages
URLs	Map URLs to views
Forms	Handle user input
Middleware	Process request and response

- ## History and Background

➢ Django was originally developed by developers working at the **Lawrence Journal-World** to manage news websites efficiently. It was publicly released in **2005** as an open-source project and is now maintained by the **Django Software Foundation**.

- ## Key Features of Django

(1) MVT Architecture

➢ Django follows the **Model-View-Template (MVT)** design pattern:

- **Model** – Defines database structure.
- **View** – Handles business logic and requests.
- **Template** – Controls presentation (HTML).

This separation improves maintainability and scalability.

(2) Rapid Development

➢ Django allows developers to build applications quickly because it includes:

- Built-in authentication system
- ORM (Object Relational Mapper)
- Admin interface
- URL routing system
- Template engine

(3) Built-in Admin Panel

- One of Django's most powerful features is the **automatic admin dashboard**.

  With minimal configuration, developers can manage:

  - Users
  - Database records
  - Application content

(4) Object Relational Mapping (ORM)

- Django ORM lets developers interact with databases using **Python code instead of SQL**.

  - **Example**:

    User.objects.filter(age__gt=18)

    Supported databases include:

    - SQLite
    - PostgreSQL
    - MySQL
    - Oracle

(5) Security

- Django includes built-in protection against common web vulnerabilities such as:

  - SQL Injection
  - Cross-Site Scripting (XSS)
  - Cross-Site Request Forgery (CSRF)
  - Clickjacking

(6) <u>Scalability</u>

- Django powers many large-scale websites and can handle high traffic with proper configuration.

  Examples include:

  - Instagram
  - Pinterest
  - Mozilla
  - The Washington Post

Q-2) Advantages of Django (e.g.,scalability, security).

- Django is widely used for web development because it provides many built-in features that make building applications **faster, secure, and scalable**. Below are some important advantages.

(1) <u>Rapid Development</u>

- Django follows the **"Don't Repeat Yourself (DRY)"** principle and provides many built-in components such as authentication, URL routing, and database management.

  Benefits

  - Less coding required
  - Faster project development
  - Easy prototyping of web applications

(2) High Scalability

- Django can handle **large-scale applications and heavy traffic** efficiently. Its modular architecture allows developers to scale different components independently.

  Large platforms such as **Instagram** and **Pinterest** use Django to manage millions of users.

(3) Strong Security

- Django provides built-in protection against many common web vulnerabilities.

  It helps prevent:

  - SQL Injection
  - Cross-Site Scripting (XSS)
  - Cross-Site Request Forgery (CSRF)
  - Clickjacking

  Security features are enabled by default, which makes web applications safer.

(4) Built-in Admin Interface

- Django automatically creates a **powerful admin dashboard** where developers can manage application data.

  Features include:

  - Add, edit, and delete database records
  - Manage users and permissions
  - Easy data administration

(5) Object Relational Mapping (ORM)

- Django includes a powerful **ORM system** that allows developers to interact with the database using Python instead of writing SQL queries.

  Example:

  User.objects.filter(age__gt=18)

  This improves productivity and reduces database errors.

(6) Clean and Organized Architecture

- Django follows the **Model–View–Template (MVT)** architecture.

  Benefits:

  - Clear separation of components
  - Easier maintenance
  - Better code readability

(7) Large Community and Documentation

- Django has a large global developer community and extensive documentation maintained by the **Django Software Foundation**.

  This provides:

  - Tutorials and guides
  - Third-party packages
  - Community support

(8) Reusable Components

- Django encourages the use of **reusable apps**, which allows developers to reuse modules across multiple projects.

  Example reusable features:

  - Authentication system
  - Form handling
  - Content management modules

(9) Supports Multiple Databases

- Django supports several popular databases including:

  - SQLite
  - PostgreSQL
  - MySQL
  - Oracle

  Developers can switch databases with minimal configuration.

Q-3) Django vs. Flask comparison: Which to choose and why

- Both **Django** and **Flask** are popular Python frameworks used for web development. However, they differ in design philosophy, features, and use cases.

(1) Basic Overview

Feature	Django	Flask
Type	Full-stack framework	Micro framework
Philosophy	Batteries-included	Minimal and flexible
Architecture	MVT (Model-View-Template)	No fixed architecture
Learning Curve	Moderate	Easy for beginners
Flexibility	Less flexible	Highly flexible

(2) Built-in Features

- **Django**

- Django comes with many built-in tools such as:

- Authentication system
- Admin dashboard
- ORM for database management
- Form handling

- Security protections

➢ This makes it ideal for **large and complex applications**.

- **Flask**

➢ Flask provides only the core functionality:

- Routing
- Request handling
- Template rendering

Additional features must be added using external libraries.

(3) Performance

Framework	Performance
Django	Slightly heavier
Flask	Lightweight and faster for small apps

(4) Security

➢ Django provides **built-in security features**, including protection from:

- SQL Injection
- Cross-Site Scripting (XSS)
- Cross-Site Request Forgery (CSRF)

- Flask requires developers to **implement security manually or with extensions**.

(5) Scalability

- Django is highly scalable and is used by large platforms such as:
  - Instagram
  - Pinterest
- Flask is also scalable but usually requires additional configuration.

(6) When to Choose Django

Choose **Django** if you are building:

- Large web applications
- Content management systems
- E-commerce websites
- Applications requiring authentication and admin panels

Reason: Django already provides many tools needed for complex projects.

(7) When to Choose Flask

Choose **Flask** if you are building:

- Small web applications
- REST APIs
- Microservices
- Lightweight projects

Reason: Flask is simple, flexible, and easy to customize.

(5) Virtual Environment

Q-1) Understanding the importance of a virtual environment in Python projects.

- A Python virtual environment is an **isolated workspace** that manages specific dependencies and Python interpreter versions for each project, preventing conflicts and ensuring consistent, reproducible results across different machines. It is considered a best practice for all Python development.

- ## Key Benefits

- **Dependency Isolation:** Different projects often require different versions of the same library (e.g., Project A needs Library 1.0, while Project B needs Library 2.0). A virtual environment keeps each project's packages separate, so they don't interfere with each other.

- **Reproducibility:** You can use `pip freeze > requirements.txt` to save an exact list of all installed packages and their versions. This file ensures that other developers or deployment servers can recreate an identical environment, guaranteeing consistent behavior.

- **Cleaner Global Environment:** Virtual environments prevent the global Python installation on your operating system from becoming cluttered with project-specific packages. This avoids unintended side effects or breaking system tools that might rely on specific, older library versions.

- **Project Portability and Collaboration:** The ability to define and recreate an environment from a `requirements.txt` file makes it

easy to share projects with team members and move them to different systems without dependency issues.

- **<u>Version Freedom</u>:** Developers can test projects with different Python versions or experimental libraries without affecting their main development setup.

- **<u>No Administrator Privileges</u>:** Packages are installed within the project's directory, so you can manage dependencies without needing special administrator or root privileges on the host system.

Q-2) Using venv or virtualenv to create isolated environments.

- Python's venv and virtualenv tools are used to create isolated environments for Python projects, preventing conflicts between different projects' dependencies.

  - **<u>How to Use venv (Recommended for Python 3.3+)</u>**

    - venv is the standard and recommended tool in modern Python.

(1) <u>Create the environment</u>:

- Open your terminal or command prompt, navigate to your project directory, and run the following command. The name venv is a common convention:

```
python -m venv venv
```

- This creates a directory named venv in your project folder.

(2) Activate the environment:

- Activation modifies your shell's PATH to use the isolated Python interpreter and packages.
  - **macOS/Linux**:
    - source venv/bin/activate
  - **Windows (Command Prompt)**:
    - venv\Scripts\activate.bat
  - **Windows (PowerShell)**:
    - .\venv\Scripts\Activate.ps1

- Once activated, your terminal prompt will typically show the environment's name in parentheses (e.g., (venv)).

(3) Install packages:

- Use pip as usual. Packages will be installed into the isolated environment.

```
pip install package-name
```

(4) Deactivate the environment:

- When you are done working on the project, simply run:

```
deactivate
```

- This returns you to your global system Python environment.

(6) Project and App Creation

Q-1) Steps to create a Django project and individual apps within the project.

- Creating a Django project and app involves a few command-line steps for setup and configuration. A project is the overall configuration for a website, while an app is a smaller, reusable module that handles a specific functionality (e.g., a blog, a polls system, a user account system).

- **Steps to Create a Django Project**

- Before starting, ensure you have Python installed and are using a virtual environment to isolate your project's dependencies.

(1) Create and Activate a Virtual Environment

- Create a directory for your projects: mkdir DjangoProjects and navigate into it: cd DjangoProjects.
- Create the virtual environment: python -m venv venv_name.
- Activate the environment (command varies by OS):
- Windows (Command Prompt): .\venv_name\Scripts\activate
- macOS/Linux: source venv_name/bin/activate

(2) Install Django

- With your environment activated, install Django using pip:

```
pip install Django
```

(3) Create the Django Project

- Run the following command to create a new project. Replace "mywebsite" with your desired project name:

```
django-admin startproject mywebsite
```

- Navigate into the newly created project directory:

```
cd mywebsite
```

(4) Run the Development Server (Optional, but recommended for verification)

- Ensure everything is working by running the server:

```
python manage.py runserver
```

- View the "Welcome to Django" page at http://127.0.0.1:8000/ in your browser. Press Ctrl+C in your terminal to stop the server.

- **Steps to Create and Configure a Django App**

(1) Create the App

- Make sure you are in the directory that contains the manage.py file.

- Run the startapp command, replacing "myapp" with your app's name:

```
python manage.py startapp myapp
```

- his creates a new myapp directory with a basic file structure, including models.py, views.py, and admin.py.

(2) Register the App in the Project

- Open the settings.py file located in your main project folder (mywebsite/settings.py).

- Add your new app's name to the INSTALLED_APPS list:

```
INSTALLED_APPS = [
# ... existing apps ...
'myapp',
]
```

(3) Define URLs for the App

- In your app's directory (myapp), create a new file named urls.py.

- In this new file, define the URL patterns for your app (e.g., mapping a view function to a URL).

- In your project's main urls.py file (mywebsite/urls.py), link to the app's urls.py using include:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('myapp/', include('myapp.urls')), # Link to your app's URLs
]
```

- Your Django project is now set up and configured to work with your new app. From here, you can start defining models, views, and templates within your app.

Q-2) Understanding the role of manage.py, urls.py, and views.py.

- In a Django project, manage.py, urls.py, and views.py play distinct, critical roles in managing the project, routing requests, and handling application logic.

- **manage.py**:
  - This is a command-line utility script automatically generated when you create a new Django project. It serves as a project-specific wrapper around the general django-admin tool, ensuring that it uses your project's correct settings.py file.

- **Role**: It is used for various administrative tasks within your project, such as:
    - Running the development server using runserver.
    - Creating new apps with startapp.
    - Managing database operations (e.g., makemigrations, migrate).
    - Running tests.
    - Creating a superuser with createsuperuser.

- **urls.py**:
    - This file acts as the "table of contents" or the URL dispatcher for your Django project or a specific app.

    - **Role**: It defines the mapping between specific URL paths requested by a user's browser and the corresponding view function that should handle that request. When a request comes in, Django checks the urlpatterns list in urls.py from top to bottom, stopping at the first pattern that matches the URL and passing the request to the associated view.

- **views.py**:
    - This file contains the core logic for handling web requests and generating web responses.

    - **Role**: A view is a Python function or class that takes a web request as its input and returns an HTTP response, which can be an HTML page (often rendered from a template), a redirect, a 404 error, or an XML document, etc.. It is where you put your application's business logic, such as interacting with the database (models) or processing data before rendering a template.

(7) MVT Pattern Architecture

Q-1) Django's MVT (Model-View-Template) architecture and how it handles request-response cycles.

- Django's MVT (Model-View-Template) architecture is a design pattern that separates application logic into three interconnected components, while the framework itself manages the control flow, similar to an MVC (Model-View-Controller) pattern where the framework acts as the controller.

- **Components of Django's MVT Architecture**

(1) Model:

- This is the data access layer, a Python class that defines the structure and schema of the database, handling data storage, maintenance, and retrieval using Django's powerful Object-Relational Mapper (ORM). It provides an interface to interact with the database without writing raw SQL queries.

(2) View:

- The view is where the core business logic and data processing happen. It is a Python function or class that receives a web request, interacts with the Model to fetch or update data, and decides which Template to render, passing the necessary context data to it. The view is responsible for returning an HTTP response object.

(3) Template:

- This is the presentation layer, typically an HTML file interspersed with the Django Template Language (DTL) syntax. The template engine uses the dynamic data provided by the view to generate the final, static HTML content presented to the user's browser.

• The Request-Response Cycle

- The entire process of handling a user's request and sending back a response involves several steps, managed by Django's internal mechanics, including middleware and the URL dispatcher.

  1. **Request Initiation**: A user action (e.g., clicking a link or submitting a form) generates an HTTP request, which is sent to the Django server.
  2. **Middleware Processing**: The request first passes through a series of middleware components. These "hooks" can modify the request object, manage sessions, perform security checks (like CSRF protection), or handle authentication before it reaches the core application logic.
  3. **URL Dispatcher**: The request is then routed to the URL dispatcher (urls.py), which matches the requested URL pattern with a specific view function.
  4. **View Execution**: The corresponding view function is called, and the HttpRequest object is passed as its first argument. The view executes the business logic, which may involve:

     4.1 Interacting with the Model to query or modify data in the database.

     4.2 Processing any form data (GET, POST, etc.) from the request.

5. **Template Rendering**: If needed, the view calls the render() function, passing the request object, the path to the Template file, and a dictionary of data (the context). Django's template engine processes the template, dynamically inserting the data from the context into the HTML.
6. **Response Generation & Middleware**: An HttpResponse object (containing the fully rendered HTML or other data like JSON) is generated and passed back through the middleware chain in reverse order for final processing (e.g., adding headers or compressing content).
7. **Response Sent**: The final response is sent back to the web server and then to the user's browser, which renders the page.

(8) Django Admin Panel

Q-1) Introduction to Django's built-in admin panel.

- Django's built-in admin panel is an automatic, model-centric interface that provides a ready-to-use, web-based dashboard for site administrators to manage content, users, and permissions without writing extra backend code. It's a key part of Django's "batteries-included" philosophy, accelerating development by automating common administrative tasks.

- **Key Features**

- **Automatic Interface**: The admin interface is automatically generated by Django based on the metadata from your defined data models, saving significant development time.
- **CRUD Operations**: It provides a full set of Create, Read, Update, and Delete (CRUD) operations for registered models, allowing administrators to interact directly with database records through a user-friendly interface.
- **User and Permission Management**: The admin panel comes with a robust, built-in system for managing users, groups, and permissions, which helps in controlling access to various parts of the application.
- **Customization**: While functional out-of-the-box, the admin interface is highly customizable. Developers can modify how models are displayed, add search boxes, filters, and even override HTML templates to match specific project requirements.
- **Internal Tool**: The Django project recommends its use for an organization's internal data management tool, not as the public-facing front end of a website.

- ## How to Access and Use

  - The admin panel is enabled by default in the INSTALLED_APPS list of a Django project's settings.py file. To access it:

(1) Create a Superuser: In your project's root directory (where manage.py is located), run the command python manage.py createsuperuser and follow the prompts to create an admin account with full permissions.

(2) Run the Server: Start the development server with python manage.py runserver.

(3) Navigate: Open your web browser and go to http://127.0.0.1:8000/admin/.

(4) Log In: Use the credentials from the superuser account you just created.

Q-2) Customizing the Django admin interface to manage database records.

- Customizing the Django admin interface is primarily done by defining a ModelAdmin class in your app's admin.py file to override the default settings for a given model. This allows you to tailor how database records are displayed, filtered, searched, and edited.

- ## Key Customization Techniques

  - **List View Customization**: Use list_display to add columns to the change list, list_filter to add filtering capabilities, and search_fields to enable search functionality.
  - **Form Layout**: Employ fieldsets to group fields logically in the edit form, and readonly_fields to prevent editing specific fields.

- **Inlines**: Use TabularInline or StackedInline to edit related models directly on the parent object's page.
- **Actions**: Define custom actions to perform bulk operations on selected objects.
- **Site Customization**: Modify admin.site.site_header, site_title, and index_title in urls.py to change the interface text.

➢ For advanced changes, you can override default Django admin templates.

(9) URL Patterns and Template Integration

Q-1) Setting up URL patterns in urls.py for routing requests to views.

- In Django, URL patterns are defined in a urls.py file within an app or project, using the urlpatterns list and the path() or re_path() functions to route requests to specific views.

- **Key Components**
  - **urls.py**: The Python module where URL patterns are defined. Django looks for a variable named urlpatterns in this file.
  - **urlpatterns**: A Python list of path() or re_path() instances.
  - **path()**: The recommended function for defining URL patterns using a simple, readable syntax with built-in path converters.
  - **re_path()**: Used for more complex patterns using regular expressions (regex).
  - **include()**: A function used to reference other URL configuration files, enabling modularity for multi-app projects.

- **Steps to Set Up URL Patterns**

(1) Define Views: Create your view functions in the views.py file of your application.

python

```
# myapp/views.py
```

```
from django.http import HttpResponse


def index(request):
    return HttpResponse("Hello, world! You're at the index.")


def detail(request, question_id):
    return HttpResponse(f"You're looking at question {question_id}.")
```

(2) <u>Create App-Level urls.py</u>: Create a urls.py file inside your app directory (e.g., myapp/urls.py) and define the patterns.

```
python
# myapp/urls.py
from django.urls import path
from . import views


urlpatterns = [
    path('', views.index, name='index'),
    path('<int:question_id>/', views.detail, name='detail'), # Dynamic URL with path converter
]
```

(3) <u>Configure Project-Level urls.py</u>: In your project's main urls.py file, use include() to point to the app's URLs.

```python
# myproject/urls.py
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('myapp/', include('myapp.urls')),
]
```

Q-2) Integrating templates with views to render dynamic HTML content.

- Integrating templates with views to render dynamic HTML involves using a template engine to combine static HTML structures with dynamic data fetched by server-side logic. This approach separates concerns, allowing for reusable components and data-driven content generation.

- **<u>Core Concept: Separation of Concerns</u>**

  - The fundamental principle is to separate the data fetching and business logic (the "view" or controller in an MVC framework) from the presentation logic (the "template").

- **Server-Side Logic (View/Controller)**: This part handles user requests, fetches necessary data (e.g., from a database), and prepares an object containing that data.
- **Template Engine**: This is a library or framework module that takes the static HTML file and the dynamic data object, replaces placeholders with actual values, and applies presentation logic (like conditional rendering and loops).
- **Dynamic HTML**: The final, complete HTML document generated by the engine is sent to the client's browser, which then renders the page.

• How It Works (General Steps)

1. **Define a Template**: Create an HTML file that includes special syntax (placeholders or tags) specific to the chosen template engine (e.g., {{ variable_name }} for Jinja2 or <%= variable_name %> for EJS).
2. **Configure the View Engine**: In your server-side application (e.g., using Express.js or Flask), configure the template engine and specify the directory where your templates are stored.
3. **Pass Data from the View**: In the route handler or controller method, you call a function (like render_template in Flask or res.render in Express/NestJS) to render the specified template and pass the data object to it.
4. **Rendering**: The template engine processes the template, injecting the data and applying any embedded logic (e.g., an if statement to check if a user is logged in), and generates the final HTML output.

- **Common Template Engines and Examples**

  - Different web development frameworks use various template engines:

  - **Node.js (Express/NestJS)**: Popular options include EJS, Pug, and Handlebars. EJS (Embedded JavaScript) allows embedding regular JavaScript logic within HTML using special tags.
  - **Python (Flask/Django)**: Flask commonly uses Jinja2, which uses double curly braces {{ ... }} for variables and {% ... %} for control flow.
  - **Angular**: Front-end frameworks like Angular use their own template syntax for binding data properties and rendering dynamic content directly in the browser.
  - **Vanilla JavaScript**: Plain JavaScript can use the native HTML <template> tag element for performance-optimized, reusable HTML structures that are cloned and inserted into the DOM dynamically.

(10) Form Validation using JavaScript

Q-1) Using JavaScript for front-end form validation.

- JavaScript is widely used for front-end form validation to enhance the user experience by providing immediate feedback and reducing server load. It allows for more sophisticated logic and custom error messages beyond basic HTML5 attributes.

- **Implementation Steps**

1. **Select Form Elements:** Get references to the form and its input fields using methods like document.getElementById() or document.querySelector().
2. **Attach an Event Listener:** Add a submit event listener to the form element.
3. **Implement Validation Logic:** Inside the event handler, define functions to validate each input field's data. This can include:
    1. **Checking for empty fields** (basic validation).
    2. **Data format validation** using regular expressions (regex) for emails, phone numbers, or passwords.
    3. **Cross-field validation**, such as confirming a password match.
4. **Display Error Messages:** If validation fails for a field, display an appropriate error message and prevent the form submission.
5. **Submit the Form (if valid):** If all validation checks pass, the form submission can proceed (or you can use AJAX to send the data to the server).

- **Example Code Snippet (JavaScript)**

  - This example demonstrates how to prevent default submission and check if a field is empty:

```
const form = document.getElementById("myForm");
const nameInput = document.getElementById("name");

form.addEventListener("submit", (event) => {
  // Prevent the form from submitting automatically
  event.preventDefault();

  if (nameInput.value.trim() === "") {
    alert("Name must be filled out");
    nameInput.focus(); // Guide user to the problematic field
    return false;
  }

  // If validation passes, the form can be submitted or an AJAX call made
  alert("Form is valid! Submitting...");
  // In a real application, you might now submit the form manually or use fetch()
});
```

(11) Django Database Connectivity (MySQL or SQLite)

Q-1) Connecting Django to a database (SQLite or MySQL).

- **Connecting Django to a Database**

➢ Django uses SQLite as its default database, requiring minimal setup and no additional software. For a scalable application, connecting to a system like MySQL is recommended.

- **Option 1: SQLite (Default)**

➢ SQLite is ready to use in any new Django project with no extra configuration needed.

1. **Verify Configuration:** In your project's settings.py file, the DATABASES dictionary is already configured for SQLite by default:

```
python
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

2. **<u>Run Migrations</u>:** Django uses migrations to create the necessary database tables. Run the following command in your terminal:

   python manage.py migrate
   This command will create the default db.sqlite3 file in your project's root directory.

- ## <u>Option 2: MySQL</u>

➢ Connecting to a MySQL database involves several steps, including installing a connector and configuring settings.py.

1. **<u>Set up MySQL Server</u>:**

➢ Ensure you have a MySQL server installed and running.

➢ Using the MySQL shell or a tool like phpMyAdmin or MySQL Workbench, create a new database and a dedicated user with all privileges granted to that database.

```
CREATE DATABASE your_database_name;

CREATE USER 'your_username'@'localhost' IDENTIFIED BY 'your_password';

GRANT ALL PRIVILEGES ON your_database_name.* TO 'your_username'@'localhost';

FLUSH PRIVILEGES;
```

2. **<u>Install MySQL Client for Python</u>:**

➢ You need a Python-compatible database connector. The most common and recommended is mysqlclient.

➢ Install it using pip in your virtual environment:

```
pip install mysqlclient
```

3. **<u>Configure Django settings.py</u>:**

- Open your project's settings.py file and replace the default DATABASES configuration with your MySQL details:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'your_database_name',
        'USER': 'your_username',
        'PASSWORD': 'your_password',
        'HOST': 'localhost', # Or the IP address of your MySQL server
        'PORT': '3306',      # Default MySQL port
    }
}
```

4. **<u>Run Migrations</u>:**

- Apply the migrations to create the default Django tables in your new MySQL database:

```
python manage.py migrate
```

5. **<u>Test the Connection</u>:**

- Start the development server to verify the connection is successful:

  python manage.py runserver

Q-2) Using the Django ORM for database queries.

- The Django ORM (Object-Relational Mapper) allows developers to interact with their database using Python code instead of writing raw SQL queries, which provides benefits like security against SQL injection, database portability, and improved code readability.

- **<u>Core Concepts</u>**
  - **<u>Models</u>:** Python classes that define the structure of your database tables. Each attribute in the model represents a database field.
  - **<u>Manager</u>:** Every model has a Manager (named objects by default) which is the interface used for all database interactions.
  - **<u>QuerySets</u>:** Represent a collection of objects from the database. They are "lazy" - meaning the query is not executed until the QuerySet is evaluated (e.g., when you iterate over it or call a function like list() or count()).

- **Common ORM Operations**

  - Assume you have a model named Book:

```
from django.db import models

class Book(models.Model):
    title = models.CharField(max_length=200)
    author_name = models.CharField(max_length=100)
    published_date = models.DateField()
```

  - **Creating Records**:

```
# Method 1: Create an object and save it
new_book = Book(title="Dune", author_name="Frank Herbert", published_date="1965-08-01")
new_book.save()
```

  - **Retrieving Records (Read)**:

  - **All records**: all_books = Book.objects.all()
  - **Filter records**: recent_books = Book.objects.filter(published_date__year__gte=2000)

- **<u>Get a single record</u>:** dune_book = Book.objects.get(title="Dune") (raises an error if no object or multiple objects are found)
- **<u>Exclude records</u>:** not_dune = Book.objects.exclude(title__contains="Dune")
- **<u>Order records</u>:** books_by_title = Book.objects.order_by('title') (use a leading hyphen for descending order)

▪ <u>Updating Records:</u>

 # Update a single record (retrieve, modify, save)

book = Book.objects.get(title="Dune")

book.author_name = "Frank Herbert Jr."

book.save()

 # Update multiple records efficiently (batch update)

Book.objects.filter(author_name="George Orwell").update(author_name="Eric Blair")

▪ <u>Deleting Records:</u>

 # Delete a single record (retrieve, delete)

book = Book.objects.get(title="Dune")

book.delete()

 # Delete multiple records efficiently

Book.objects.filter(author_name="Eric Blair").delete()

(12) ORM and QuerySets

Q-1) Understanding Django's ORM and how QuerySets are used to interact with the database.

- Django's Object-Relational Mapper (ORM) is a powerful system that allows developers to interact with the database using Python code instead of raw SQL. The primary interface for this interaction is the QuerySet, which represents a collection of database queries and acts as a list of Python objects from your models.

- **How the Django ORM and QuerySets Work**

1. **Models as Tables**: Django models are Python classes that map directly to database tables. Each attribute in the class represents a column in that table.
2. **Managers as Interfaces**: Every model has at least one Manager (by default named objects) which is the interface used to retrieve objects from the database.
3. **QuerySets as Queries**: When you call methods on a manager (like all(), filter(), etc.), you are creating a QuerySet. A QuerySet represents a potential SQL query but doesn't run it immediately.
4. **Lazy Evaluation:** QuerySets are "lazy". This means the database is not actually hit until the QuerySet is evaluated (e.g., when you iterate over it in a template or a Python loop, call len(), or use specific methods like list() or count()). This allows for efficient chaining of multiple filter and order conditions without numerous database calls.

5. **Translation to SQL**: Behind the scenes, Django's ORM translates the Python QuerySet methods into the appropriate, secure SQL queries for your specific database backend (SQLite, PostgreSQL, MySQL, etc.).

(13) Django Forms and Authentication

Q-1) Using Django's built-in form handling.

- Django's built-in form handling simplifies the complex process of web forms by managing HTML generation, data validation, sanitization, error handling, and security (specifically CSRF protection). This framework helps centralize form logic, following the "Don't Repeat Yourself" (DRY) principle.

- **Core Components**

  - Django form handling relies on several key components:

- **Form Class**: The fundamental component for defining form fields and their behaviors.
- **ModelForm Class**: A powerful shortcut that automatically generates a form from an existing Django model, simplifying the process of saving data to the database.

- **Views**: Python functions or class-based views that handle the logic for displaying the form, processing submitted data, and redirecting the user.
- **Templates**: Use Django's template tags (e.g., {{ form.as_p }}) to render the form's HTML elements easily.
- **{% csrf_token %}**: A crucial security tag required in all POST forms to prevent Cross-Site Request Forgery (CSRF) attacks.

- ## Step-by-Step Handling Process

  - The standard process for handling a form in a Django view involves both GET and POST requests:

1. **Initial Request (GET)**: The user requests the page, triggering a GET request. The view instantiates an empty, unbound form and passes it to the template for rendering.

```
def my_view(request):
    if request.method == 'POST':
        # Handle POST data here
    else:
        form = MyForm() # Unbound form
    return render(request, 'my_template.html', {'form': form})
```

2. **Form Submission (POST)**: The user fills out the form and submits it, sending a POST request to the same view (typically).
3. **Data Binding and Validation**: In the view, a new form instance is created and "bound" to the request.POST data. The form.is_valid() method is called to check the data against the

field's defined validation rules (e.g., email format, max length, date range).

```
def my_view(request):
    if request.method == 'POST':
        form = MyForm(request.POST) # Bound form
        if form.is_valid():
            # Process the data
            # ...
            # Redirect after successful submission
    else:
        form = MyForm()
    return render(request, 'my_template.html', {'form': form})
```

4. **<u>Processing Valid Data</u>**: If is_valid() returns True, the cleaned, sanitized data is available in the form.cleaned_data dictionary. This data is then used for tasks such as saving a model instance to the database or sending an email.

```
if form.is_valid():
    # Example for ModelForm
    form.save() # Saves directly to the database
    # Example for standard Form
    name = form.cleaned_data['name']
    email = form.cleaned_data['email']
    # ... perform actions ...
```

```
return HttpResponseRedirect('/success_url/') # Redirect to a new URL
```

5. **Handling Invalid Data**: If is_valid() returns False, Django automatically re-renders the form, pre-populating the fields with the user's submitted data and displaying relevant error messages.

Q-2) Implementing Django's authentication system (sign up, login, logout, password management).

- Implementing Django's authentication system is a streamlined process using its built-in tools. You primarily need to connect the provided views in your URL configuration and create the necessary templates.

- **Setup**

1. **Verify INSTALLED_APPS**:
   Ensure 'django.contrib.auth' and 'django.contrib.contenttypes' are in your INSTALLED_APPS in settings.py (they are included by default in new projects).

2. **Configure Redirect URLs**: In settings.py, define where users are redirected after logging in or logging out.

```
LOGIN_REDIRECT_URL = '/' # Redirect to home page after login
LOGOUT_REDIRECT_URL = '/login/' # Redirect to login page after logout
```

3. **<u>Set up Templates Directory</u>**: Ensure your TEMPLATES setting includes the path to your main templates directory in settings.py.

```
TEMPLATES = [
    {
        ...
        'DIRS': [os.path.join(BASE_DIR, 'templates')],
        ...
    },
]
```

- **<u>URL Configuration</u>**

  - In your project's main urls.py, include the built-in authentication URLs:

```
from django.contrib.auth import views as auth_views
from django.urls import path, include
# ... other imports

urlpatterns = [
    # ... other paths
    path('accounts/', include('django.contrib.auth.urls')),
]
```

- This line automatically provides URLs for login/, logout/, password_change/, password_reset/, and their "done" views.

- **Sign Up (Registration)**

  - Since Django does not provide a built-in registration view, you must create it.

- **Form**: Utilize UserCreationForm from django.contrib.auth.forms to create a SignUpForm.
- **View**: Create a sign_up view that handles the form, validates it, and logs the user in upon success using django.contrib.auth.login.
- **Template**: Render this form in a template, often named registration/sign_up.html.

- **Login, Logout, and Password Management**

  - Django offers built-in class-based views for login, logout, and password management (change, reset) that require specific template naming conventions in templates/registration/.

- **Login/Logout**: The LoginView looks for registration/login.html, while LogoutView handles sign-out.
- **Password**: Specialized views like PasswordChangeView and PasswordResetView handle password functionality.

(14) CRUD Operations using AJAX

Q-1) Using AJAX for making asynchronous requests to the server without reloading the page.

- AJAX (Asynchronous JavaScript and XML) is a set of web development techniques using various web technologies on the client-side to create asynchronous web applications. It allows web pages to be updated asynchronously by exchanging small amounts of data with the server behind the scenes, thereby avoiding the need to reload the entire page.

- **How AJAX Works:**

1. **Event Triggered:** A user event (e.g., button click, form submission) or a programmatic event triggers a JavaScript function.
2. **XMLHttpRequest Object Created:** The JavaScript function creates an XMLHttpRequest object.
3. **Request Sent:** The XHR object sends an asynchronous request to the server, specifying a URL and a method (e.g., GET or POST).
4. **Server Processes Request:** The server-side script receives the request, processes the data (e.g., retrieves data from a database), and generates a response.
5. **Response Received:** The XHR object receives the server's response. A callback function in the JavaScript is executed to handle this response.
6. **DOM Updated:** The JavaScript uses the DOM to dynamically update the relevant part of the web page with the new data, without a full page reload.

(15) Customizing the Django Admin Panel

Q-1) Techniques for customizing the Django admin panel.

- Customizing the Django admin panel can be done through two primary methods: modifying the ModelAdmin class in admin.py or by overriding the default admin templates. Third-party packages offer simpler ways to apply themes.

- **Customization via ModelAdmin Class (in admin.py)**

  - The ModelAdmin class is the primary way to control how your models appear and behave in the admin interface.

- **list_display**: Controls which fields are displayed as columns on the change list page (the page that lists all objects of a model). You can include model fields, callables, or methods on the model or ModelAdmin.
- **list_filter**: Adds a sidebar for filtering the change list by specific fields (e.g., date ranges, boolean values, or related fields using __ lookups).
- **search_fields**: Adds a search box to the change list page that searches the specified fields using OR queries.
- **fields and fieldsets**: Controls the layout, order, and grouping of fields on the "add" and "change" forms. fieldsets allows more complex organization with titles and CSS classes.
- **readonly_fields**: Specifies fields that are read-only when editing an object.

- **Inlines**: Allows you to edit related model objects on the same change page as the parent object (e.g., editing OrderItems when editing an Order). This
uses admin.TabularInline or admin.StackedInline.
- **Custom Actions**: You can define and register custom functions that perform actions on selected objects from the change list page.

- **Customization via Template Overriding**

  - For more extensive visual or functional changes, you can override the default admin templates. Django uses a specific template loading order, which allows you to place custom templates in your project's templates/admin directory to replace the default ones.

  - **base_site.html**: The main template for the admin site, often overridden to change the branding, header, or title.
  - **index.html**: The template for the admin homepage (dashboard).
  - **change_form.html / change_list.html**: Used to customize the object editing and listing pages, respectively.

(16) Payment Integration Using Paytm

Q-1) Introduction to integrating payment gateways (like Paytm) in Django projects

- Integrating the Paytm payment gateway in a Django project involves a server-side SDK/API approach that handles secure transactions through a series of steps including account setup, API key management, transaction initiation, and callback verification.

- **Prerequisites**
  - **Paytm Merchant Account:** Register for a business account on the Paytm for Business website to obtain your Merchant ID (MID) and Secret Key. You can use their "sandbox" environment for testing first.
  - **Django Project:** A functional Django project with a basic structure (apps, templates, URLs).
  - **Python Libraries:** You'll need Django and a library for checksum generation (e.g., pycryptodome, as the older pycrypto is deprecated).

- **Integration Steps**

1. **Install Dependencies:**
   Install necessary libraries in your Django project's virtual environment:

```
pip install django pycryptodome
```

- You may also need to incorporate the specific checksum utility file provided in the Paytm documentation or tutorials that is compatible with pycryptodome.

2. Configure Django Settings:

- Add your Paytm credentials to your settings.py file. It is best practice to use environment variables for sensitive data.

```
# settings.py
PAYTM_MERCHANT_ID = 'your_merchant_id'
PAYTM_SECRET_KEY = 'your_paytm_secret_key'
PAYTM_WEBSITE = 'WEBSTAGING' # Use "WEBSTAGING" for testing, "DEFAULT" for live
PAYTM_CHANNEL_ID = 'WEB'
PAYTM_INDUSTRY_TYPE_ID = 'Retail'
PAYTM_CALLBACK_URL = 'http://127.0.0.1:8000/callback/' # Define your callback URL
```

3. Create Payment Models:

- Define a model in your Django app (e.g., payments/models.py) to store transaction details, such as order ID, amount, and status, to track payments.

4. Implement Payment Views:

- Create views to handle the payment lifecycle:

  1. **Initiate Payment View:** This view prepares the transaction parameters (MID, Order ID, Amount, Customer ID, Callback URL, etc.), generates a unique checksum hash

using the Paytm library, saves the transaction data in your model, and renders a redirect page.

2. **Redirect Page Template:** A simple HTML page that uses JavaScript to automatically submit a POST form with all the payment parameters (including the checksum hash) to the Paytm payment gateway URL (e.g., https://securegw-stage.paytm.in/order/process/ for staging).

5. Handle the Callback:

Create a callback view (at the URL specified in PAYTM_CALLBACK_URL) that receives a POST request from Paytm's server after the transaction is complete. In this view:

1. Retrieve the received data.
2. Verify the CHECKSUMHASH using the Paytm library's verification function to ensure data integrity and prevent forged requests.
3. Update your transaction model's status based on the response (success/failure/pending).

6. Define URLs:

Map your views to specific URLs in your urls.py file.

7. Testing:

Thoroughly test the integration in the staging/sandbox environment using the provided test credentials and simulated payment methods before going live. Ensure all scenarios, including success, failure, and pending transactions, are handled correctly.

(17) GitHub Project Deployment

Q-1) Steps to push a Django project to GitHub.

- To push a Django project to GitHub, you need to use Git commands in your project's terminal to initialize a local repository, link it to a new remote GitHub repository, and then push the files.

- **Prerequisites**
  - A GitHub account (GitHub homepage).
  - Git installed on your local machine (Git Downloads).
  - A created Django project (and virtual environment activated).

- **Step-by-Step Guide**

1. Prepare your Django Project

- Navigate to your project's root directory in your terminal (the folder containing manage.py).

- **Create a requirements.txt file**: This file lists all project dependencies, allowing others (or your deployment server) to install them later.

```
pip freeze > requirements.txt
```

2. Initialize Git Locally and Commit

- **Initialize a local Git repository** in your project root:

  git init

- **Stage** all the relevant project files:

  git add .

- **Commit** the staged files with a descriptive message:

  git commit -m "Initial commit of Django project"

3. Create a GitHub Repository

- Go to the GitHub website and sign in.
- Click the **+** icon in the top-right corner and select **New repository**.
- Give your repository a unique name (ideally the same as your project name), add an optional description, and choose between Public or Private visibility.
- **Do not** check the options to add a README, .gitignore, or license, as you have already created these locally.
- Click **Create repository**.
- On the next screen, copy the repository's HTTPS URL (e.g., https://github.com/yourusername/yourrepo.git).

4. Link Local to Remote and Push

- **Link** your local repository to the remote GitHub repository using the copied URL:

git remote add origin https://github.com/yourusername/yourrepo.git

- (Optional but recommended) **Rename** your default local branch to match GitHub's default branch name (main):

  git branch -M main

- **Push** your local code to the main branch on GitHub:

  git push -u origin main

If this is your first time pushing from your machine, you will be prompted to sign in to GitHub.

5. Verify on GitHub

- Refresh your repository page on the GitHub website. Your Django project files should now be visible in the repository.

(18) Live Project Deployment (PythonAnywhere)

Q-1) Introduction to deploying Django projects to live servers like PythonAnywhere.

- Deploying a Django project involves preparing your code for a production environment and configuring a web server to run it. PythonAnywhere simplifies this by providing a tailored environment and a user-friendly dashboard for configuration.

- The general process for deploying a Django project to PythonAnywhere is:

1. Prepare Your Django Project Locally

- Before moving to the server, make these essential changes in your local settings.py file:

- **DEBUG = False**: Never run a production site with DEBUG set to True.
- **ALLOWED_HOSTS**: Add your site's domain name to the list (e.g., ['yourusername.pythonanywhere.com']).
- **SECRET_KEY**: Use environment variables to keep sensitive information like your SECRET_KEY private, rather than hardcoding it in your settings.py file.
- **Static Files**: Configure STATIC_URL and STATIC_ROOT settings. Run python manage.py collectstatic to gather all static files into a single directory for the server to serve them efficiently.

- **requirements.txt**: Create a requirements.txt file using pip freeze > requirements.txt to list all project dependencies.

2. Set Up Your PythonAnywhere Account and Code

- **Create an Account**: Sign up for a PythonAnywhere account (the free beginner account is sufficient for a basic deployment).
- **Upload/Clone Code**: Use the Files tab to upload a zip file of your project, or use a Bash console to clone your project directly from a source code repository like GitHub using git clone [your-repo-url].
- **Create Virtual Environment**: In a Bash console, create a virtual environment to install your project's specific dependencies.

```
mkvirtualenv --python=python3.X my-project-venv
```

(Replace python3.X with your desired Python version).

- **Install Requirements**: Navigate to your project directory in the Bash console and install the dependencies:

```
pip install -r requirements.txt
```

3. Configure the Web App

- **Add Web App**: Go to the Web tab on your dashboard and click Add a new web app. Choose the Manual configuration option, selecting the same Python version you used for your virtual environment.

- **Configure Paths**: On the web app configuration page, provide the correct paths:
  - **Virtualenv**: Enter the path to your newly created virtual environment (e.g., /home/yourusername/.virtualenvs/my-project-venv).
  - **Source code**: The path to your project's top folder (the one containing manage.py).

- **Edit WSGI File**: Click the link to the WSGI configuration file (it will be named something like /var/www/yourusername_pythonanywhere_com_wsgi.py). Edit the file to configure it for your specific Django project by uncommenting and updating the Django-specific lines to point to your project's settings.

```
import os
import sys

# Add your project path to the sys.path
path = '/home/yourusername/your_project_folder'
if path not in sys.path:
    sys.path.append(path)

os.environ['DJANGO_SETTINGS_MODULE'] = 'your_project_folder.settings' # Replace 'your_project_folder' with the name of the directory containing settings.py

from django.core.wsgi import get_wsgi_application
application = get_wsgi_application()
```

- Set up Static Files: In the Web tab, find the Static files section. Add a mapping for STATIC_URL (e.g., /static/) and the full path to your STATIC_ROOT directory.

4. Finalize and Go Live

- Database Migrations: Go back to your Bash console, navigate to your project directory, and run any necessary database migrations:

```
python manage.py migrate
```

- **Reload**: Return to the Web tab and hit the Reload button for your website.

- **Test**: Click the link to your site (e.g., http://yourusername.pythonanywhere.com) to see your application live. Check the error log links on the Web tab if you encounter any issues.

(19) Social Authentication

Q-1) Setting up social login options (Google, Facebook, GitHub) in Django using OAuth2.

- Setting up social login (Google, Facebook, GitHub) in Django using OAuth2 is most effectively done using the django-allauth library, which abstracts much of the OAuth2 complexity.

Step 1: Install and Configure django-allauth

1. **Install**: pip install django-allauth.
2. **Settings (INSTALLED_APPS)**:
   Add 'django.contrib.sites', 'allauth', 'allauth.account', 'allauth.social account', and the desired provider packages (e.g., 'allauth.socialaccount.providers.google').
3. **Authentication Backends**:
   Include allauth.account.auth_backends.AuthenticationBackend in s ettings.py.
4. **Configuration**: Set SITE_ID = 1 and add allauth.urls to your urls.py.
5. **Migrate**: Run python manage.py migrate.

Step 2: Configure Provider OAuth Apps

- Register your app on developer portals (Google, Facebook, GitHub) to obtain Client IDs and Secrets. Set the callback URL to http://localhost:8000/accounts/[provider]/login/callback/.

Step 3: Add Credentials in Django Admin

1. Navigate to 127.0.0.1:8000/admin/.
2. Set the **Site** domain to localhost:8000 or 127.0.0.1:8000.
3. Under **Social Applications**, add new entries for each provider using the credentials from Step 2, associating them with the site.

Step 4: Add Login Links

- Use {% load socialaccount %} and {% provider_login_url '[provider]' %} in your HTML templates to create login buttons.

(20) Google Maps API

Q-1) Integrating Google Maps API into Django projects.

- Integrating the Google Maps API into a Django project involves a mix of backend configuration to securely manage the API key and frontend development using JavaScript to display and interact with the map.

Step 1: Set Up Google Cloud Project and API Key

1. **Create a Project**: Go to the Google Cloud Platform Console and create a new project.
2. **Enable APIs**: Enable the specific APIs your project needs, such as the Maps JavaScript API, Geocoding API, and Directions API.
3. **Generate API Key**: Navigate to the "Credentials" page and create a new API key. Restrict this key to prevent unauthorized use in production.
4. **Add Key to Django Settings**: Store your API key securely in your Django project's settings.py file, ideally using environment variables for production.

```
# settings.py
import os

GOOGLE_MAPS_API_KEY =
os.environ.get('GOOGLE_MAPS_API_KEY', 'YOUR_API_KEY_HERE')
```

Step 2: Backend (Django Views and URLs)

- You need to pass the API key to the frontend template that will render the map.

1. **Define a View**: In your app's views.py, create a view to render your map template, passing the API key in the context.

```
# your_app/views.py
from django.shortcuts import render
from django.conf import settings

def map_view(request):
    context = {'google_maps_api_key':
settings.GOOGLE_MAPS_API_KEY}
    return render(request, 'map.html', context)
```

2. **Map the URL**: In your app's urls.py, define a URL pattern for the view.

```
# your_app/urls.py
from django.urls import path
from . import views

urlpatterns = [
    path('map/', views.map_view, name='map_view'),
]
```

Step 3: Frontend (HTML Template and JavaScript)

- The map is primarily handled using the Google Maps JavaScript API within your HTML template.

1. **Create the Template**: In your templates/map.html file, include a element where the map will be displayed and a <script> tag to load the Google Maps API.

```
<!-- templates/map.html -->
<!DOCTYPE html>
<html>
<head>
    <style>
        #map {
            height: 400px;
            width: 100%;
        }
    </style>
</head>
<body>
    <h2>My Django Google Map</h2>
    <div id="map"></div>

    <!-- Load the API with the key from Django context -->
    <script src="https://maps.googleapis.com{{
google_maps_api_key }}&callback=initMap" async defer></script>
```

```
        <script>
            function initMap() {
                // The location of the default center point (e.g., New York)
                const center = { lat: 40.730610, lng: -73.935242 };
                // The map, centered at the specified location
                const map = new
google.maps.Map(document.getElementById("map"), {
                    zoom: 12,
                    center: center,
                });
                // Optional: Place a marker
                new google.maps.Marker({
                    position: center,
                    map: map,
                    title: "Default Location",
                });
            }
        </script>
    </body>
    </html>
```